

Universidade Federal de Minas Gerais

Automação em Tempo Real

Sistema de Inspeção Automática de Túneis

Relatório da Etapa 1

Leandro Lopo – Matrícula 2021020520

Rodrigo Martins – Matrícula 2023421459

Professor: Leandro Freitas

Belo Horizonte
18 de maio de 2026

Sumário

1	Introdução	1
2	Proposta de Arquitetura	1
2.1	Diagrama da Arquitetura	2
2.2	Interações Planejadas Entre Processos	3
2.3	Tarefas Implementadas	3
2.4	Buffers e Estados Compartilhados	4
2.5	Mecanismos de Sincronização	4
3	Implementação Desenvolvida	5
3.1	Tipos de Dados	5
3.2	Buffers Sincronizados	6
3.3	Simulação dos Sensores	7
3.4	Consumo com Variável de Condição	8
3.5	Reconstrução da Superfície	8
3.6	Detecção de Falha	9
3.7	Distância Percorrida	9
3.8	Coletor de Dados	10
3.9	Controle de Navegação	10
3.10	Inspecção por Câmera	11
3.11	Criação das Threads	12
3.12	Compilação	13
4	Resultados de Testes Preliminares	13
4.1	Teste 1: Troca de Dados entre Simulação e Reconstrução	14
4.2	Teste 2: Cálculo de Distância Percorrida	14
4.3	Teste 3: Coletor de Dados e Arquivo CSV	14
4.4	Teste 4: Detecção de Falha e Acionamento da Câmera	15
4.5	Teste 5: Comando e Controle de Navegação	15
4.6	Análise dos Testes	16
5	Conclusão	16

1 Introdução

Este trabalho desenvolve a arquitetura inicial de um sistema de inspeção automática de túneis. O robô se desloca horizontalmente, mede a distância até o teto com um LIDAR, reconstrói o perfil da superfície e detecta variações bruscas que indicam possíveis buracos ou saliências.

Na Etapa 1, o objetivo é definir a arquitetura completa do projeto e implementar o núcleo interno do robô em C++. Essa implementação inclui threads, buffers compartilhados, mutexes, variáveis de condição e testes preliminares para verificar se as tarefas trocam dados de forma coerente.

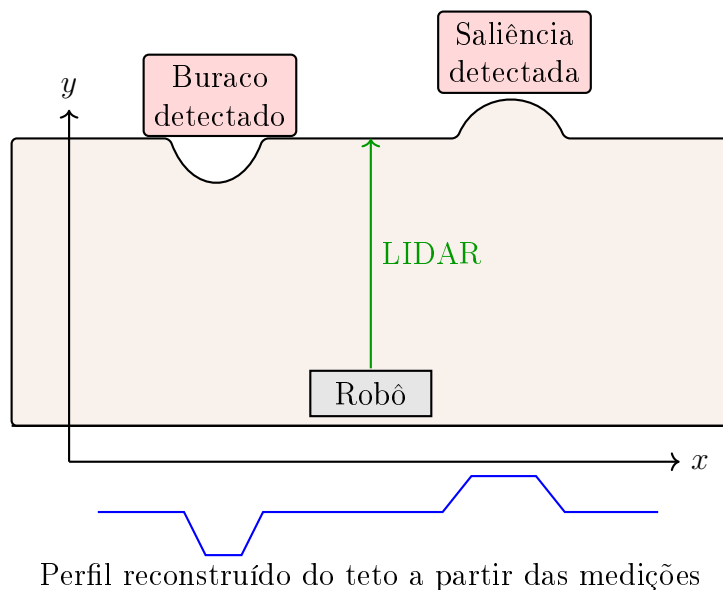


Figura 1: Ilustração própria do problema tratado: robô inspecionando o teto de um túnel em vista lateral 2D.

2 Proposta de Arquitetura

A arquitetura proposta cobre o projeto completo, e não apenas o subconjunto implementado nesta primeira entrega. O sistema foi dividido em dois níveis: um núcleo embarcado em C++, responsável pelas tarefas de tempo real do robô, e processos externos para simulação, operação remota e comunicação publisher/subscriber. Essa separação acompanha a organização indicada no enunciado e deixa claro quais partes pertencem ao controle interno do robô e quais partes serão integradas por IPC na etapa seguinte.

No núcleo do robô, as tarefas internas foram modeladas como threads C++ dentro de um único processo. A comunicação entre essas threads ocorre por memória compartilhada, com buffers protegidos por `std::mutex` e `std::condition_variable`. Essa decisão é adequada para a Etapa 1 porque permite demonstrar diretamente os conceitos de concorrência, região crítica, produtor-consumidor e sincronização.

Os processos externos de simulação, operação remota e broker MQTT foram especificados na arquitetura completa, mas não implementados nesta etapa. Na Etapa 1, a thread `SimulacaoSensores` substitui temporariamente a interface de simulação para alimentar os buffers internos e permitir testes preliminares. Na Etapa 2, essa thread de teste deve ser removida ou substituída pela entrada de dados recebida via MQTT.

2.1 Diagrama da Arquitetura

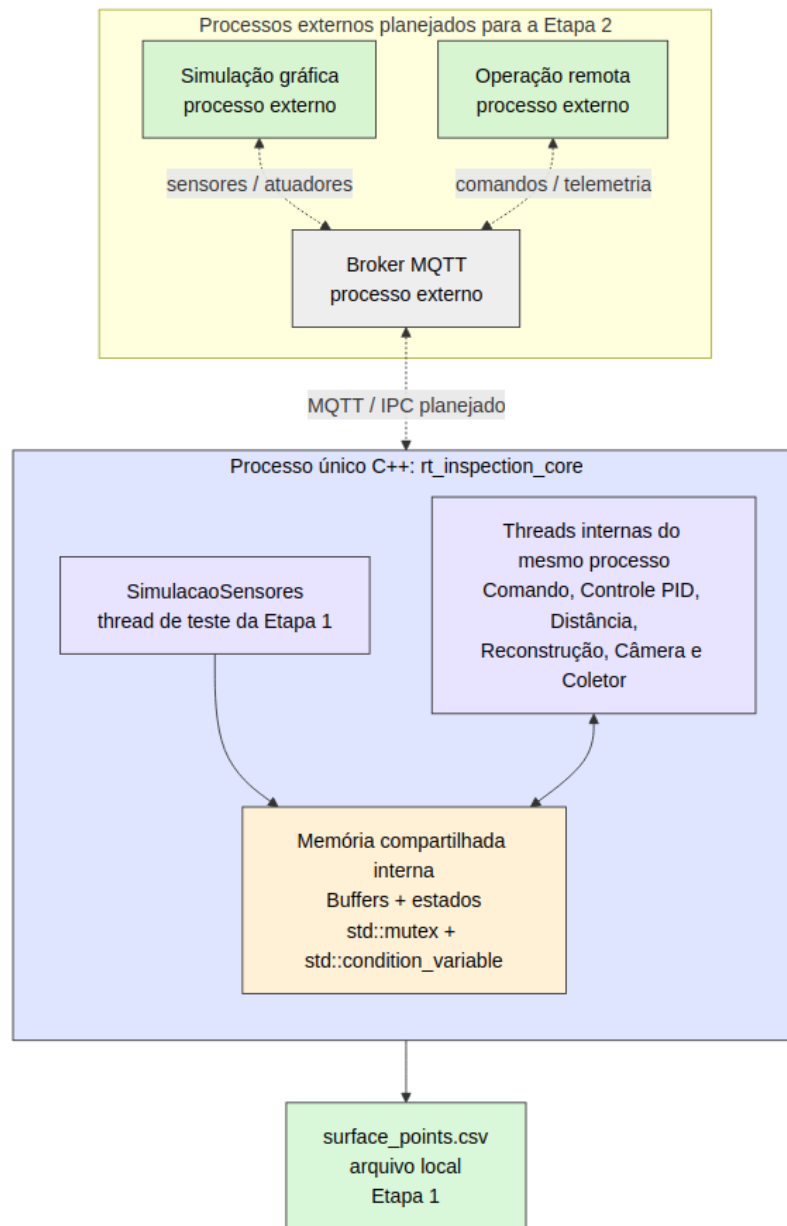


Figura 2: Diagrama de contêineres da arquitetura completa.

A Figura 2 usa uma visão de contêineres, semelhante ao modelo C4 usado em documentação de sistemas. Ela separa processos externos, broker MQTT, núcleo C++ e armazenamento local. No diagrama, as setas tracejadas representam interfaces MQTT planejadas para a Etapa 2, enquanto as setas contínuas representam fluxos internos ou saída local do núcleo C++ na Etapa 1. O detalhamento interno do núcleo é apresentado na Figura 3.

2.2 Interações Planejadas Entre Processos

As interações externas do projeto completo serão implementadas por MQTT na Etapa 2. O broker atua como intermediário entre processos, seguindo o modelo publisher/subscriber. A Tabela 1 resume os fluxos planejados.

Tabela 1: Comunicação planejada entre processos externos e núcleo do robô.

Origem	Destino	Mecanismo	Dados
Simulação gráfica	Núcleo C++	MQTT: sim/sensors	Leituras de LIDAR e encoder.
Núcleo C++	Simulação gráfica	MQTT: core/actuators	Aceleração e comando de câmera.
Operação remota	Núcleo C++	MQTT: remote/commands	Modo manual/automático e setpoints.
Núcleo C++	Operação remota	MQTT: core/state	Posição, velocidade e estado de inspeção.
Núcleo C++	Operação remota	MQTT: core/surface	Pontos reconstruídos e confiança.

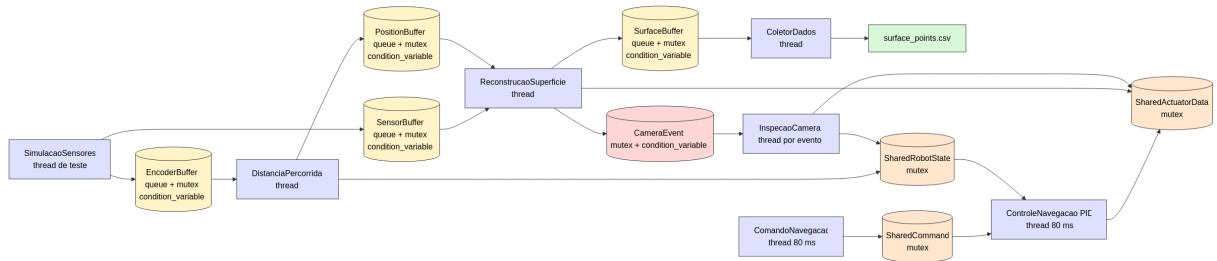


Figura 3: Diagrama de componentes e fluxo de dados do núcleo C++ implementado na Etapa 1.

2.3 Tarefas Implementadas

Tarefa	Tipo	Responsabilidade
SimulacaoSensores	Thread C++ de teste	Gera dados simulados de LIDAR e encoder para testar os buffers da Etapa 1.
ReconstrucaoSuperficie	Thread C++	Consome leituras de LIDAR e posições calculadas, aplica média móvel, gera pontos de superfície e detecta anomalias.

Tarefa	Tipo	Responsabilidade
DistanciaPercorrida	Thread C++	Consome o encoder, calcula a posição horizontal do robô e publica amostras de posição.
ColetorDados	Thread C++	Consome pontos reconstruídos, calcula confiança online e grava os dados em CSV.
ComandoNavegacao	Thread C++ cíclica	Simula comando de operação remota na Etapa 1, configurando modo automático e setpoint.
ControleNavegacao	Thread C++ cíclica	Aplica controle PID simples e gera aceleração.
InspecaoCamera	Thread C++ por evento	Aguarda sinal de falha e executa processamento pesado usando CPU.

2.4 Buffers e Estados Compartilhados

Os buffers implementam o padrão produtor-consumidor. Cada buffer possui uma fila, um mutex, uma variável de condição e uma flag de finalização.

Estrutura	Produtor	Consumidor	Sincronização
SensorBuffer	SimulacaoSensores	Reconstrucao Superficie	mutex_sensor dado_disp.
EncoderBuffer	SimulacaoSensores	Distancia Percorrida	mutex_encoder encoder_disp.
PositionBuffer	Distancia Percorrida	Reconstrucao Superficie	mutex_posicao posicao_disp.
SurfaceBuffer	Reconstrucao Superficie	ColetorDados	mutex_superficie surface_point_var
CameraEvent	Reconstrucao Superficie	InspecaoCamera	mutex_camera camera_event_var

Além dos buffers, foram utilizados estados compartilhados protegidos por mutex:

- **SharedRobotState**: armazena posição, velocidade, modo automático e estado de inspeção.
- **SharedCommand**: armazena comandos simulados para a Etapa 1.
- **SharedActuatorData**: armazena aceleração e comando de câmera.
- **SharedSystemControl**: controla o encerramento das tarefas cíclicas.

2.5 Mecanismos de Sincronização

O `std::mutex` é usado para proteger regiões críticas, como inserção e remoção de dados em filas ou leitura e escrita em estados compartilhados. A regra usada no código foi manter o mutex travado apenas durante o acesso ao dado compartilhado, copiando o valor para uma variável local e realizando o processamento fora da região crítica.

A `std::condition_variable` é usada para evitar espera ocupada. As tarefas consumidoras ficam bloqueadas até que uma tarefa produtora insira um novo dado no buffer e

chame `notify_one()`. Essa estratégia reduz uso desnecessário de CPU e evita laços de polling.

3 Implementação Desenvolvida

Esta seção apresenta os principais trechos de código do sistema e a justificativa das decisões tomadas em cada parte da implementação. Os trechos foram escolhidos por representarem os mecanismos centrais do trabalho: definição dos dados, comunicação por buffers, sincronização, reconstrução da superfície, detecção de falhas, controle de navegação e acionamento da câmera.

3.1 Tipos de Dados

O primeiro passo da implementação foi definir estruturas simples para representar as mensagens trocadas entre as tarefas. Essa escolha evita passar vários parâmetros soltos entre threads e deixa explícito qual informação circula em cada buffer. `SensorData` representa a leitura combinada de LIDAR e encoder, `EncoderData` separa o fluxo usado pela tarefa de distância, `PositionData` carrega a posição calculada a partir do encoder, e `SurfacePoint` representa o ponto reconstruído do teto.

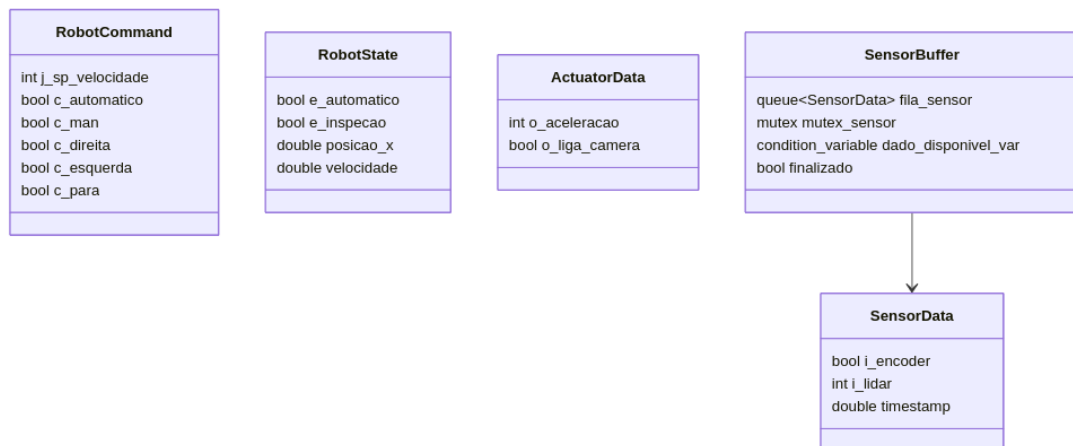


Figura 4: Diagrama UML simplificado das estruturas de comando, estado, atuadores e buffers de sensores.

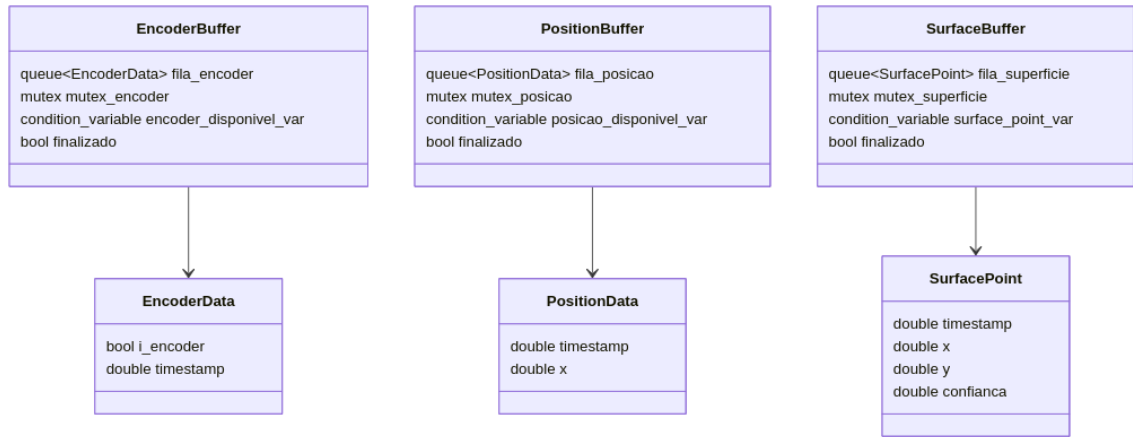


Figura 5: Diagrama UML simplificado dos buffers de posição e superfície e das mensagens associadas.

As Figuras 4 e 5 resumem a relação entre as estruturas usadas no código. Os trechos seguintes mostram as definições essenciais que sustentam a troca de dados entre as tarefas.

```

1 struct SensorData {
2     bool i_encoder;
3     int i_lidar;
4     double timestamp;
5 };
6
7 struct EncoderData {
8     bool i_encoder;
9     double timestamp;
10 };
11
12 struct PositionData {
13     double timestamp;
14     double x;
15 };
16
17 struct SurfacePoint {
18     double timestamp;
19     double x;
20     double y;
21     double confianca;
22 };
  
```

Separar `EncoderData` de `SensorData` foi importante para evitar que duas tarefas consumidoras retirassem dados da mesma fila. A reconstrução consome o fluxo de sensores e também recebe a posição calculada pela tarefa de distância através de `PositionBuffer`. Isso evita que a coordenada horizontal de um ponto reconstruído dependa do instante de escalonamento de outra thread.

3.2 Buffers Sincronizados

Os buffers foram implementados seguindo o padrão produtor-consumidor. Cada buffer possui uma fila, um mutex, uma variável de condição e uma flag de finalização. Essa estrutura foi escolhida porque permite que uma tarefa produza dados em seu próprio ritmo e outra tarefa consuma esses dados sem espera ocupada.


```

1 struct SensorBuffer {
2     std::queue<SensorData> fila_sensor;
3     std::mutex mutex_sensor;
4     std::condition_variable dado_disponivel_var;
5     bool finalizado = false;
6 };
7
8 struct SurfaceBuffer {
9     std::queue<SurfacePoint> fila_superficie;
10    std::mutex mutex_superficie;
11    std::condition_variable surface_point_var;
12    bool finalizado = false;
13 };
14
15 struct PositionBuffer {
16     std::queue<PositionData> fila_posicao;
17     std::mutex mutex_posicao;
18     std::condition_variable posicao_disponivel_var;
19     bool finalizado = false;
20 };

```

O mutex protege o acesso à fila. A `condition_variable` permite que a tarefa consumidora durma enquanto não há dados disponíveis. A flag `finalizado` evita que a consumidora fique bloqueada indefinidamente ao final da simulação.

3.3 Simulação dos Sensores

A tarefa de simulação foi usada na Etapa 1 para substituir temporariamente a interface gráfica de simulação que será implementada na Etapa 2. Ela gera leituras falsas, alterna o encoder e injeta uma anomalia no LIDAR para testar a detecção de falhas.

```

1 for (int i = 0; i < 10; i++) {
2     estado_encoder = !estado_encoder;
3
4     SensorData leitura;
5     leitura.i_encoder = estado_encoder;
6     leitura.i_lidar = (i == 5) ? 140 : 100 + i;
7     leitura.timestamp = TempoAtualSegundos();
8
9     EncoderData encoder{leitura.i_encoder, leitura.timestamp};
10
11    {
12        std::lock_guard<std::mutex> trava(encoderBuffer.mutex_encoder);
13        encoderBuffer.fila_encoder.push(encoder);
14    }
15    encoderBuffer.encoder_disponivel_var.notify_one();
16
17    {
18        std::lock_guard<std::mutex> trava(sensorBuffer.mutex_sensor);
19        sensorBuffer.fila_sensor.push(leitura);
20    }
21    sensorBuffer.dado_disponivel_var.notify_one();
22
23    std::this_thread::sleep_for(std::chrono::milliseconds(500));
24 }

```

A leitura anômala `i_lidar = 140` foi colocada para simular uma variação brusca no teto. O uso de `sleep_for` aqui representa o intervalo entre leituras de sensores simulados. Isso é diferente da tarefa de câmera, que não deve ser simulada apenas com `sleep`, pois o enunciado exige processamento pesado.

3.4 Consumo com Variável de Condição

As tarefas consumidoras usam `condition_variable` para aguardar dados. Essa decisão evita que a thread execute um laço verificando a fila repetidamente, o que desperdiçaria CPU.

```
1 SensorData leitura;
2
3 {
4     std::unique_lock<std::mutex> trava(buffer.mutex_sensor);
5
6     buffer.dado_disponivel_var.wait(trava, [&buffer] {
7         return !buffer.fila_sensor.empty() || buffer.finalizado;
8     });
9
10    if (buffer.fila_sensor.empty() && buffer.finalizado) {
11        break;
12    }
13
14    leitura = buffer.fila_sensor.front();
15    buffer.fila_sensor.pop();
16 }
```

O processamento é feito fora da região crítica. Assim, o mutex fica travado apenas pelo tempo necessário para retirar o dado da fila, reduzindo contenção entre threads.

3.5 Reconstrução da Superfície

A reconstrução aplica uma média móvel de três amostras no LIDAR. Essa técnica foi escolhida por ser simples, online e suficiente para reduzir ruídos iniciais sem exigir armazenamento de todo o histórico.

```
1 ultimasLeituras.push(leitura.i_lidar);
2 somaLeituras += leitura.i_lidar;
3
4 if (ultimasLeituras.size() > tamanhoJanela) {
5     somaLeituras -= ultimasLeituras.front();
6     ultimasLeituras.pop();
7 }
8
9 double media_movel =
10     static_cast<double>(somaLeituras) / ultimasLeituras.size();
11
12 SurfacePoint ponto;
13 ponto.timestamp = leitura.timestamp;
14 ponto.x = posicao.x;
15 ponto.y = media_movel;
16 ponto.confianca = 1.0;
```

A coordenada x vem do `PositionBuffer`, produzido pela tarefa de distância percorrida. Dessa forma, cada ponto reconstruído combina a leitura vertical filtrada com a posição horizontal correspondente à mesma amostra de sensor.

3.6 Detecção de Falha

A falha é detectada comparando a média móvel atual com a média anterior. A regra foi escolhida por ser simples de testar e por representar a ideia central do problema: buracos e saliências aparecem como variações bruscas no perfil do teto.

```
1  const double variacao = std::abs(media_movel - mediaAnterior);
2
3  if (variacao > limiteFalha && !falhaJaDetectada) {
4      falhaJaDetectada = true;
5
6      {
7          std::lock_guard<std::mutex> trava(robotState.mutex_estado);
8          robotState.estado.e_inspecao = true;
9      }
10
11     {
12         std::lock_guard<std::mutex> trava(sharedActuatorData.
13             mutex_atuadores);
14         sharedActuatorData.atuadores.o_liga_camera = true;
15     }
16
17     {
18         std::lock_guard<std::mutex> trava(cameraEvent.mutex_camera);
19         cameraEvent.falha_detectada = true;
20         cameraEvent.timestamp = ponto.timestamp;
21         cameraEvent.x = ponto.x;
22         cameraEvent.y = ponto.y;
23     }
24
25     cameraEvent.camera_event_var.notify_one();
26 }
```

Quando a falha é encontrada, a reconstrução atualiza o estado do robô, liga a câmera e sinaliza o evento que acorda a tarefa `InspecaoCamera`. A flag `falhaJaDetectada` evita múltiplos disparos para a mesma anomalia simulada.

3.7 Distância Percorrida

A tarefa `DistanciaPercorrida` implementa a regra do enunciado: o encoder troca de estado a cada metro percorrido. Sempre que o valor atual do encoder difere do anterior, a posição horizontal é incrementada. Além de atualizar `SharedRobotState`, a tarefa publica a posição em `PositionBuffer`, permitindo que a reconstrução receba uma posição coerente com cada leitura de LIDAR.

```
1  bool houveMudanca = leitura.i_encoder != encoderAnterior;
2
3  if (leitura.i_encoder != encoderAnterior) {
4      encoderAnterior = leitura.i_encoder;
5  }
6
7  PositionData posicao;
```

```

8 posicao.timestamp = leitura.timestamp;
9
10 {
11     std::lock_guard<std::mutex> trava(robotState.mutex_estado);
12
13     if (houveMudanca) {
14         robotState.estado.posicao_x += 1.0;
15     }
16
17     posicao.x = robotState.estado.posicao_x;
18 }
19
20 {
21     std::lock_guard<std::mutex> trava(positionBuffer.mutex_posicao);
22     positionBuffer.fila_posicao.push(posicao);
23 }
24 positionBuffer.posicao_disponivel_var.notify_one();

```

O estado do robô foi centralizado em `SharedRobotState` para que a posição também possa ser usada por outras tarefas. O `PositionBuffer` foi adicionado especificamente para manter a ordem temporal entre a leitura do sensor e a posição usada na reconstrução.

3.8 Coletor de Dados

O coletor recebe os pontos reconstruídos e grava o arquivo CSV. Além disso, calcula uma confiança simples com base na quantidade de pontos próximos já observados. Essa escolha atende ao requisito de análise online, mantendo a implementação simples para a Etapa 1.

```

1 int pontosProximos = 1;
2
3 for (const SurfacePoint &anterior : historico) {
4     const bool proximoEmX = std::abs(anterior.x - ponto.x) <= 2.0;
5     const bool proximoEmY = std::abs(anterior.y - ponto.y) <= 15.0;
6
7     if (proximoEmX && proximoEmY) {
8         pontosProximos++;
9     }
10 }
11
12 ponto.confianca = std::min(1.0, pontosProximos / 5.0);
13 historico.push_back(ponto);
14
15 arquivo << ponto.timestamp << ", "
16         << ponto.x << ", "
17         << ponto.y << ", "
18         << ponto.confianca << "\n";

```

O CSV permite verificar o perfil reconstruído e serve como evidência objetiva dos testes realizados.

3.9 Controle de Navegação

O controle de navegação foi implementado como um PID simples. Essa escolha aproxima a implementação do comportamento esperado para uma malha de controle real, sem esconder a lógica principal em bibliotecas externas. O controlador lê o setpoint pro-

duzido pela tarefa `ComandoNavegacao`, compara com a velocidade atual armazenada em `SharedRobotState` e escreve a aceleração em `SharedActuatorData`.

```
1  const double kp = 25.0;
2  const double ki = 1.5;
3  const double kd = 4.0;
4  const double periodoSegundos = 0.08;
5  const double limiteIntegral = 20.0;
6
7  const double setpointVelocidade =
8      emInspecao ? std::min(static_cast<double>(comando.j_sp_velocidade),
9                          1.0)
10                  : static_cast<double>(comando.j_sp_velocidade);
11
12 const double erro = setpointVelocidade - velocidadeAtual;
13 integralErro = std::clamp(integralErro + erro * periodoSegundos,
14                          -limiteIntegral, limiteIntegral);
15 const double derivadaErro = (erro - erroAnterior) / periodoSegundos;
16 erroAnterior = erro;
17
18 const double saidaPid = kp * erro + ki * integralErro + kd *
19     derivadaErro;
20 const int aceleracao = static_cast<int>(
21     std::clamp(saidaPid, -100.0, 100.0));
22
23 {
24     std::lock_guard<std::mutex> trava(sharedActuatorData.mutex_atuadores);
25     sharedActuatorData.atuadores.o_aceleracao = aceleracao;
26     sharedActuatorData.atuadores.o_liga_camera = emInspecao;
27 }
```

Os ganhos foram mantidos conservadores para evitar oscilações grandes na simulação preliminar. O termo integral é limitado para evitar acúmulo excessivo de erro, e a saída do controlador é saturada no intervalo de aceleração permitido. Quando o robô entra em inspeção, o setpoint é limitado para reduzir a velocidade. Essa decisão acompanha o comportamento pedido no enunciado: ao detectar falha, o robô deve navegar mais devagar e acionar a câmera.

3.10 Inspeção por Câmera

A tarefa de câmera é acionada por evento. Ao acordar, ela executa um laço computacional com funções matemáticas, simulando processamento pesado de imagem. Essa escolha foi feita porque o enunciado especifica que essa tarefa deve usar o processador e não apenas dormir.

```
1  cameraEvent.camera_event_var.wait(trava, [&cameraEvent] {
2      return cameraEvent.falha_detectada || cameraEvent.finalizado;
3  });
4
5  volatile double resultado = 0.0;
6  for (int i = 0; i < 8000000; i++) {
7      resultado += std::sin(i * 0.001) * std::cos(i * 0.0005);
8  }
9
10 {
```

```

11     std::lock_guard<std::mutex> trava(sharedActuatorData.mutex_atuadores
12         );
13     sharedActuatorData.atuadores.o_liga_camera = false;
14 }

```

O uso de evento evita que a câmera execute continuamente. Ela permanece bloqueada e só consome CPU quando uma falha é detectada.

3.11 Criação das Threads

O programa principal instancia todos os buffers e estados compartilhados, cria as threads e aguarda sua finalização. Essa organização deixa o main responsável apenas pela montagem do sistema, mantendo a lógica de cada tarefa em seu próprio arquivo.

```

1  SensorBuffer sensorBuffer;
2  EncoderBuffer encoderBuffer;
3  PositionBuffer positionBuffer;
4  SurfaceBuffer surfaceBuffer;
5  CameraEvent cameraEvent;
6  SharedRobotState robotState;
7  SharedCommand sharedCommand;
8  SharedActuatorData sharedActuatorData;
9  SharedSystemControl systemControl;
10
11 {
12     std::lock_guard<std::mutex> trava(sharedCommand.mutex_comando);
13     sharedCommand.comando.c_automatico = true;
14     sharedCommand.comando.j_sp_velocidade = 2;
15 }
16
17 std::thread comando(ComandoNavegacao,
18     std::ref(sharedCommand), std::ref(robotState), std::ref(
19         systemControl));
20
21 std::thread controle(ControleNavegacao,
22     std::ref(sharedCommand), std::ref(robotState),
23     std::ref(sharedActuatorData), std::ref(systemControl));
24
25 std::thread simulacao(SimulacaoSensores,
26     std::ref(sensorBuffer), std::ref(encoderBuffer));
27
28 std::thread distancia(DistanciaPercorrida,
29     std::ref(encoderBuffer), std::ref(positionBuffer), std::ref(
30         robotState));
31
32 std::thread reconstrucao(ReconstrucaoSuperficie,
33     std::ref(sensorBuffer), std::ref(positionBuffer), std::ref(
34         surfaceBuffer),
35     std::ref(robotState), std::ref(sharedActuatorData), std::ref(
36         cameraEvent));
37
38 std::thread coletor(ColetorDados, std::ref(surfaceBuffer));
39 std::thread camera(InspecaoCamera,
40     std::ref(cameraEvent), std::ref(robotState), std::ref(
41         sharedActuatorData));

```

3.12 Compilação

A compilação foi automatizada por um Makefile. Essa escolha reduz erros na apresentação, facilita os testes e garante que todos os arquivos da pasta `src/` sejam compilados com as mesmas opções.

```
CXX := g++
CXXFLAGS := -std=c++17 -Wall -Wextra -pthread -Iinclude
BUILD_DIR := build
TARGET := $(BUILD_DIR)/rt_inspection
SRCS := $(wildcard src/*.cpp)

all: $(TARGET)

$(TARGET): $(SRCS)
    mkdir -p $(BUILD_DIR)
    $(CXX) $(CXXFLAGS) $(SRCS) -o $(TARGET)

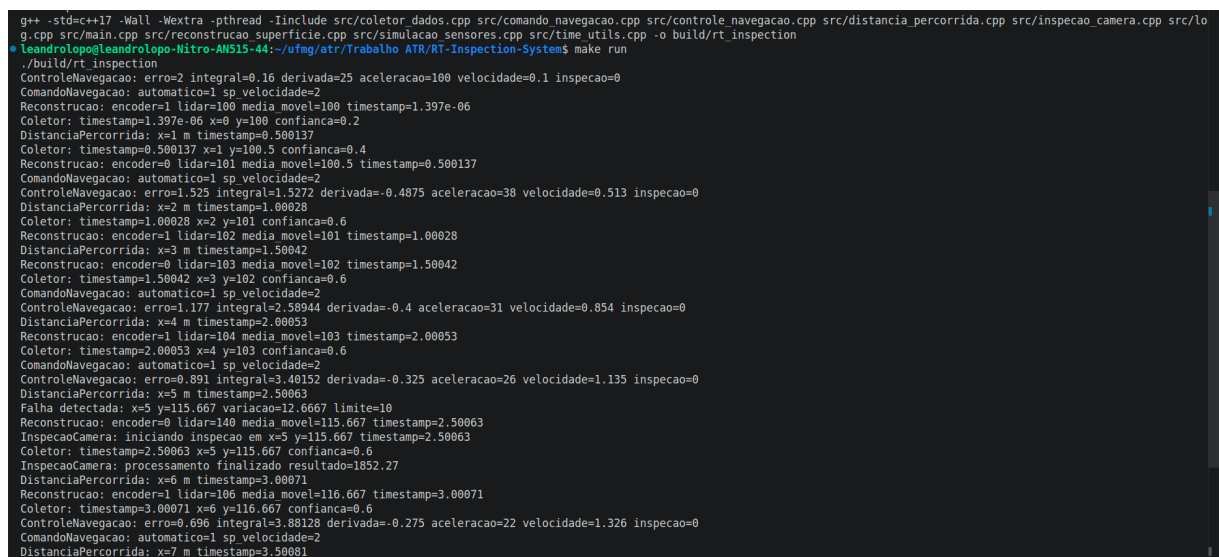
run: $(TARGET)
    ./$(TARGET)
```

4 Resultados de Testes Preliminares

Os testes foram executados com os seguintes comandos:

```
make clean
make
make run
```

A Figura 6 mostra uma execução completa do programa pelo terminal. A saída reúne mensagens de reconstrução, distância percorrida, coleta de dados, comando, controle e inspeção por câmera, evidenciando que as tarefas executam de forma concorrente e encerram sem bloqueio observado.



```
g++ -std=c++17 -Wall -Wextra -pthread -Iinclude src/coletor_dados.cpp src/comando_navegacao.cpp src/controle_navegacao.cpp src/distancia_percorrida.cpp src/inspecao_camera.cpp src/lo
g.cpp src/main.cpp src/reconstrucao_superficie.cpp src/simulacao_sensores.cpp src/time_utils.cpp -o build/rt_inspection
leandrolopo@leandrolopo-Nitro-AN515-44:~/ufmg/atr/Trabalho ATR/RT-Inspection-System$ make run
./build/rt_inspection
ControleNavegacao: erro=2 integral=0.16 derivada=25 aceleracao=100 velocidade=0.1 inspecao=0
ComandoNavegacao: automatico=1 sp velocidade=2
Reconstrucao: encoder=1 lidar=100 media_movel=100 timestamp=1.397e-06
Coletor: timestamp=1.397e-06 x=0 y=100 confianca=0.2
DistanciaPercorrida: x=1 m timestamp=0.500137
Coletor: timestamp=0.500137 x=1 y=100.5 confianca=0.4
Reconstrucao: encoder=0 lidar=101 media_movel=100.5 timestamp=0.500137
ComandoNavegacao: automatico=1 sp velocidade=2
ControleNavegacao: erro=1.525 integral=1.5272 derivada=-0.4875 aceleracao=38 velocidade=0.513 inspecao=0
DistanciaPercorrida: x=2 m timestamp=1.00028
Coletor: timestamp=1.00028 x=2 y=101 confianca=0.6
Reconstrucao: encoder=1 lidar=102 media_movel=101 timestamp=1.00028
DistanciaPercorrida: x=3 m timestamp=1.50042
Reconstrucao: encoder=0 lidar=103 media_movel=102 timestamp=1.50042
Coletor: timestamp=1.50042 x=3 y=102 confianca=0.6
ComandoNavegacao: automatico=1 sp velocidade=2
ControleNavegacao: erro=1.177 integral=2.58944 derivada=-0.4 aceleracao=31 velocidade=0.854 inspecao=0
DistanciaPercorrida: x=4 m timestamp=2.00053
Reconstrucao: encoder=1 lidar=104 media_movel=103 timestamp=2.00053
Coletor: timestamp=2.00053 x=4 y=103 confianca=0.6
ComandoNavegacao: automatico=1 sp velocidade=2
ControleNavegacao: erro=0.891 integral=3.40152 derivada=-0.325 aceleracao=26 velocidade=1.135 inspecao=0
DistanciaPercorrida: x=5 m timestamp=2.50063
Falha detectada: x=5 y=115.667 variacao=12.6667 limite=10
Reconstrucao: encoder=0 lidar=140 media_movel=115.667 timestamp=2.50063
InspecaoCamera: iniciando inspecao em x=5 y=115.667 timestamp=2.50063
Coletor: timestamp=2.50063 x=5 y=115.667 confianca=0.6
InspecaoCamera: processamento finalizado resultado=1852.27
DistanciaPercorrida: x=6 m timestamp=3.00071
Reconstrucao: encoder=1 lidar=106 media_movel=116.667 timestamp=3.00071
Coletor: timestamp=3.00071 x=6 y=116.667 confianca=0.6
ControleNavegacao: erro=0.096 integral=3.88128 derivada=-0.275 aceleracao=22 velocidade=1.326 inspecao=0
ComandoNavegacao: automatico=1 sp velocidade=2
DistanciaPercorrida: x=7 m timestamp=3.50081
```

Figura 6: Print do terminal com a execução completa do sistema.

4.1 Teste 1: Troca de Dados entre Simulação e Reconstrução

O primeiro teste verificou se a tarefa `SimulacaoSensores` produz leituras de sensores e se a tarefa `ReconstrucaoSuperficie` consome essas leituras corretamente. O resultado esperado é a impressão de leituras com o valor filtrado por média móvel. A saída observada é mostrada na Figura 7.

```
$ grep "^Reconstrucao:" logs/saida_completa.txt | head -n 6
Reconstrucao: encoder=1 lidar=100 media_movel=100 timestamp=1.467e-06
Reconstrucao: encoder=0 lidar=101 media_movel=100.5 timestamp=0.500108
Reconstrucao: encoder=1 lidar=102 media_movel=101 timestamp=1.00021
Reconstrucao: encoder=0 lidar=103 media_movel=102 timestamp=1.50033
Reconstrucao: encoder=1 lidar=104 media_movel=103 timestamp=2.00046
Reconstrucao: encoder=0 lidar=140 media_movel=115.667 timestamp=2.5006
```

Figura 7: Print do terminal do Teste 4.1: troca de dados entre simulação e reconstrução.

Esse teste valida o funcionamento do `SensorBuffer`, do mutex associado e da variável de condição usada para acordar a reconstrução.

4.2 Teste 2: Cálculo de Distância Percorrida

O segundo teste verificou se o encoder é processado pela tarefa `DistanciaPercorrida`. Como o encoder simulado troca de estado a cada leitura, a posição horizontal é incrementada quando ocorre uma mudança. A saída observada é mostrada na Figura 8.

```
$ grep "^DistanciaPercorrida:" logs/saida_completa.txt | head -n 6
DistanciaPercorrida: x=1 m timestamp=0.500108
DistanciaPercorrida: x=2 m timestamp=1.00021
DistanciaPercorrida: x=3 m timestamp=1.50033
DistanciaPercorrida: x=4 m timestamp=2.00046
DistanciaPercorrida: x=5 m timestamp=2.5006
DistanciaPercorrida: x=6 m timestamp=3.0007
```

Figura 8: Print do terminal do Teste 4.2: cálculo da distância percorrida a partir do encoder.

Esse teste valida o `EncoderBuffer`, o uso de `SharedRobotState` para manter a posição global e o `PositionBuffer` usado para entregar posições à reconstrução na mesma ordem das leituras.

4.3 Teste 3: Coletor de Dados e Arquivo CSV

O terceiro teste verificou se a reconstrução gera pontos de superfície e se o coletor consome esses pontos, calcula confiança e grava o arquivo `surface_points.csv`. A saída observada é mostrada na Figura 9.


```
$ head -n 8 surface_points.csv
timestamp,x,y,confianca
1.467e-06,0,100,0.2
0.500108,1,100.5,0.4
1.00021,2,101,0.6
1.50033,3,102,0.6
2.00046,4,103,0.6
2.5006,5,115.667,0.6
3.0007,6,116.667,0.6
```

Figura 9: Print do terminal do Teste 4.3: verificação do arquivo CSV gerado pelo coletor de dados.

Esse teste valida o `SurfaceBuffer`, o cálculo online de confiança e a gravação dos dados em disco.

4.4 Teste 4: Detecção de Falha e Acionamento da Câmera

O quarto teste inseriu uma anomalia no LIDAR simulado, configurando uma leitura com valor mais alto que o padrão. A reconstrução compara a média móvel atual com a média anterior e sinaliza falha quando a variação passa do limite configurado. A saída observada é mostrada na Figura 10.

```
leandrolopo@leandrolopo-Nitro-AN515-44:~/ufmg/atr/Trabalho ATR/RT-Inspection-Syst
em$ cat logs/teste_4_4_terminal.txt
$ grep -E "Falha detectada|InspecaoCamera" logs/saida_completa.txt
Falha detectada: x=5 y=115.667 variacao=12.6667 limite=10
InspecaoCamera: iniciando inspecao em x=5 y=115.667 timestamp=2.5006
InspecaoCamera: processamento finalizado resultado=1852.27
leandrolopo@leandrolopo-Nitro-AN515-44:~/ufmg/atr/Trabalho ATR/RT-Inspection-Syst
em$
```

Figura 10: Print do terminal do Teste 4.4: detecção de falha e acionamento da inspeção por câmera.

Esse teste valida o `CameraEvent`, a sincronização por evento e o processamento pesado da tarefa de câmera. A câmera executa um laço com funções matemáticas, usando CPU de fato, e não apenas uma chamada de `sleep`.

4.5 Teste 5: Comando e Controle de Navegação

O quinto teste verificou se as tarefas cíclicas de comando e controle executam em paralelo com as demais tarefas. O comando fixa o modo automático e o setpoint de velocidade, enquanto o controle PID calcula a aceleração a partir do erro de velocidade. A saída observada é mostrada na Figura 11.

```

leandrolopo@leandrolopo-Nitro-AN515-44:~/ufmg/atr/Trabalho ATR/RT-Inspection-Syst
em$ cat logs/teste_4_5_terminal.txt
$ grep -E "ComandoNavegacao|ControleNavegacao" logs/saida_completa.txt | head -n
8
ComandoNavegacao: automatico=1 sp_velocidade=2
ControleNavegacao: erro=2 integral=0.16 derivada=25 aceleracao=100 velocidade=0.1
inspecao=0
ComandoNavegacao: automatico=1 sp_velocidade=2
ControleNavegacao: erro=1.525 integral=1.5272 derivada=-0.4875 aceleracao=38 velo
cidade=0.513 inspecao=0
ControleNavegacao: erro=1.177 integral=2.58944 derivada=-0.4 aceleracao=31 veloci
dade=0.854 inspecao=0
ComandoNavegacao: automatico=1 sp_velocidade=2
ComandoNavegacao: automatico=1 sp_velocidade=2
ControleNavegacao: erro=0.891 integral=3.40152 derivada=-0.325 aceleracao=26 velo
cidade=1.135 inspecao=0

```

Figura 11: Print do terminal do Teste 4.5: execução das tarefas de comando e controle de navegação.

Esse teste valida o uso de `SharedCommand`, `SharedRobotState`, `SharedActuatorData` e `SharedSystemControl`. Também mostra que as tarefas cíclicas encerram corretamente após a finalização das tarefas de simulação, reconstrução, coleta e câmera.

4.6 Análise dos Testes

Durante os testes, as threads finalizaram corretamente, sem bloqueio observado. Os buffers possuem flags de finalização para evitar que consumidores fiquem bloqueados indefinidamente. As impressões no terminal são protegidas por um mutex global de log, evitando mensagens misturadas. O `PositionBuffer` remove a dependência do escalonamento entre distância e reconstrução, pois a reconstrução passa a consumir a posição calculada na mesma ordem das leituras de sensor.

5 Conclusão

Nesta etapa foi implementado o núcleo embarcado do sistema de inspeção automática de túneis. As principais tarefas foram modeladas como threads C++ dentro de um único processo. Foram implementados buffers sincronizados para comunicação entre tarefas, estados compartilhados protegidos por mutex, reconstrução do perfil do teto por média móvel, cálculo de distância percorrida pelo encoder, envio ordenado de posição para reconstrução, controle PID simples, detecção de falha por variação brusca e acionamento de inspeção por câmera via evento.

Os testes preliminares indicam que a arquitetura implementada consegue trocar dados entre tarefas sem condições de corrida aparentes, sem bloqueios observados e com finalização controlada das threads. O sistema também gera um arquivo CSV com os pontos reconstruídos, permitindo verificar o perfil estimado do teto.

Como próximos passos para a Etapa 2, devem ser implementadas as interfaces gráficas de simulação e operação remota, a comunicação entre processos via MQTT, a integração com um broker publisher/subscriber e uma análise mais detalhada de tempo de resposta e escalonabilidade das tarefas cíclicas.